

HW_06

Due: 5:00pm, 29 September

Assignment

Create a self-contained R script or Rmarkdown file to do the following:

- Fit a distance sampling model with a half-normal detection function and the following covariates to the black-throated blue warbler data (`btbw_data_distsamp.csv`) in `unmarked` and `JAGS`:
 - Density covariates: `Elevation`, `UTM.N`, `UTM.W`
 - Detection covariates: `Wind`, `Noise`
 - Response: `btbw0_20`, `btbw20_40`, `btbw40_60`, `btbw60_80`, `btbw80_100`
- Using the model fitted in `unmarked`, create two graphs of the predictions: one for density and the other for the scale parameter (σ).

Setup

```
library(dplyr)

##
## Attaching package: 'dplyr'

## The following objects are masked from 'package:stats':
##
##   filter, lag

## The following objects are masked from 'package:base':
##
##   intersect, setdiff, setequal, union

data <- read.csv("btbw_data_distsamp.csv")
data <- data %>%
  rename(
    btbw0_20 = X.0.20.,
    btbw20_40 = X.20.40.,
    btbw40_60 = X.40.60.,
    btbw60_80 = X.60.80.,
    btbw80_100 = X.80.100.,
  )

head(data)
```

	X	UTM.N	UTM.W	Elevation	btbw0_20	btbw20_40	btbw40_60	btbw60_80
## 1	PCCow001	3879223	275815	1340	0	4	0	0
## 2	PCCow002	3879727	274813	1325	4	3	0	0
## 3	PCCow003	3879727	275312	1258	0	2	0	3
## 4	PCCow004	3879724	275812	1169	0	3	0	0
## 5	PCCow005	3879727	276313	1067	0	0	1	0
## 6	PCCow006	3879727	276813	1208	0	0	0	2

	btbw80_100	SurvDate	SurvTime	Observer	Precipitation	Wind	Noise
## 1	0	2019-05-21	07:03:00	1	0	1	0
## 2	0	2019-05-21	07:36:00	1	0	0	1
## 3	0	2019-05-21	08:37:00	1	0	2	1
## 4	0	2019-05-21	10:53:00	1	0	2	1
## 5	0	2019-05-21	09:52:00	1	0	0	1
## 6	0	2019-05-22	06:27:00	1	0	0	0

unmarked

```
library(unmarked)

# extract covariates and standardize
covs <- data.frame(Elevation = data$Elevation,
                   UTM_N = data$UTM.N,
                   UTM_W = data$UTM.W,
                   Wind = data$Wind,
                   Noise = data$Noise
                   )

covs_standard <- covs %>%
  mutate(across(where(is.numeric), ~ as.vector(scale(.))))

# extract response var as matrix
y <- data %>%
  select(btbw0_20,
         btbw20_40,
         btbw40_60,
         btbw60_80,
         btbw80_100)
y <- as.matrix(y)

# 'dist.breaks' for 'unmarkedFrameDS'
b <- c(0, 20, 40, 60, 80, 100)

# fit model in 'unmarked'
umf <- unmarkedFrameDS(y = y,
                      siteCovs = covs_standard,
                      dist.breaks = b,
                      survey = "point",
                      unitsIn = "m")

# display summary
summary(umf)
```

```

## unmarkedFramedDS Object
##
## point-transect survey design
## Distance class cutpoints (m):  0 20 40 60 80 100
##
## 109 sites
## Maximum number of distance classes per site: 5
## Mean number of distance classes per site: 5
## Sites with at least one detection: 51
##
## Tabulation of y observations:
##   0   1   2   3   4   5   7   8  12
## 448 28 27 17 21  1  1  1  1
##
## Site-level covariates:
##      Elevation      UTM_N      UTM_W      Wind
## Min.    :-1.6301  Min.    :-1.8824  Min.    :-1.9411  Min.    :-0.8446
## 1st Qu.: -0.9063  1st Qu.: -0.7712  1st Qu.: -0.8440  1st Qu.: -0.8446
## Median : -0.1500  Median :  0.1690  Median :  0.2499  Median :  0.1144
## Mean    :  0.0000  Mean    :  0.0000  Mean    :  0.0000  Mean    :  0.0000
## 3rd Qu.:  0.7152  3rd Qu.:  0.9074  3rd Qu.:  0.7162  3rd Qu.:  1.0733
## Max.    :  1.9505  Max.    :  2.0171  Max.    :  2.0334  Max.    :  2.0322
##      Noise
## Min.    :-1.13497
## 1st Qu.: -1.13497
## Median : -0.08657
## Mean    :  0.00000
## 3rd Qu.:  0.96184
## Max.    :  3.05866

fm <- distsamp(~Noise+Wind ~ UTM_N+UTM_W+Elevation, umf, keyfun = "halfnorm")
fm

##
## Call:
## distsamp(formula = ~Noise + Wind ~ UTM_N + UTM_W + Elevation,
## data = umf, keyfun = "halfnorm")
##
## Density (log-scale):
##      Estimate      SE      z P(>|z|)
## (Intercept)   0.903 0.1195  7.55 4.20e-14
## UTM_N         -0.259 0.0907 -2.86 4.28e-03
## UTM_W         -0.756 0.0969 -7.81 5.76e-15
## Elevation      0.468 0.0862  5.42 5.85e-08
##
## Detection (log-scale):
##      Estimate      SE      z P(>|z|)
## (Intercept)   3.4313 0.0367 93.495 0.00000
## Noise          0.0298 0.0302  0.987 0.32382
## Wind          -0.0707 0.0250 -2.825 0.00472
##
## AIC: 1000.445
## Number of sites: 109
##

```

```
## Survey design: point-transect
## Detection function: halfnorm
## UnitsIn: m
## UnitsOut: ha
```

JAGS

State model (Poisson)

$$\log(\lambda_i) = \beta_0 + \beta_1 w_{i1} + \beta_2 w_{i2} + \dots$$

$$N_i \sim \text{Poisson}(\lambda_i)$$

Observation Model

$$\log(\sigma_i) = \alpha_0 + \alpha_1 w_{i1} + \alpha_2 w_{i3} + \dots$$

$$\{y_{i1}, \dots, y_{iK}\} \sim \text{Multinomial}(N_i, \pi(x, \sigma_i))$$

```
model_string <- "
model {

  beta0 ~ dnorm(0, 0.1)
  beta1 ~ dnorm(0, 0.1)
  beta2 ~ dnorm(0, 0.1)
  beta3 ~ dnorm(0, 0.1)

  alpha0 ~ dnorm(0, 0.1)
  alpha1 ~ dnorm(0, 0.1)
  alpha2 ~ dnorm(0, 0.1)

  for(i in 1:nSites) {
    log(lambda[i]) <- beta0 + beta1*Elevation[i] + beta2*UTM_N[i] + beta3*UTM_W[i]
    N[i] ~ dpois(lambda[i]*Area)

    log(sigma[i]) <- alpha0 + alpha1*Noise[i] + alpha2*Wind[i]
    for(j in 1:nBins) {
      pbar[i,j] <- (sigma[i]^2 * (1-exp(-b[j+1]^2/(2*sigma[i]^2)))) -
                    sigma[i]^2 * (1-exp(-b[j]^2/(2*sigma[i]^2)))) *
                    2*3.141593/area[j]
      pi[i,j] <- psi[j]*pbar[i,j]
    }
    pi[i,nBins+1] <- 1-sum(pi[i,1:nBins]) ## Pr(not detected)
    n[i] ~ dbin(1-pi[i,nBins+1], N[i])
    y[i,] ~ dmulti(pi[i,1:nBins]/(1-pi[i,nBins+1]), n[i])
    ## If N~Pois(lam), then the above is equivalent to:
    # for(j in 1:nBins) { y[i,j] ~ dpois(lambda[i]*pi[i,j]) }
  }

  totalAbundance <- sum(N[1:nSites])

}
"
# make tempfile so model string can be referenced without file
model_file <- tempfile(fileext = ".jags")
```

```

writeLines(model_string,
           con = model_file
           )

B <- 100
Area <- base::pi *B^2/1e4
nSites <- nrow(y)
J <- ncol(y)
b <- seq(0,B, length=J+1)
area <- base::pi *b^2
psi <- (area[-1]-area[-(J+1)]) / area[J+1]

jags_data <- list(y=y,
                 n=rowSums(y),
                 area=diff(area),
                 b=b,           # Distance break points
                 Area=Area,     # Area in ha
                 psi=psi,       # Pr(occurring in bin j)
                 Elevation=covs_standard$Elevation,
                 UTM_N=covs_standard$UTM_N,
                 UTM_W=covs_standard$UTM_W,
                 Noise=covs_standard$Noise,
                 Wind=covs_standard$Wind,
                 nSites=nSites,
                 nBins=J
                 )

jags_inits <- function() {
  list(beta0 = rnorm(1),
        alpha0 = log(20),
        N = rowSums(y) + rpois(nrow(y),2)
        )
}

jags_pars <- c("beta0",
               "beta1",
               "beta2",
               "beta3",
               "alpha0",
               "alpha1",
               "alpha2",
               "totalAbundance"
               )

library(jagsUI)
jags_post <- jags.basic(data=jags_data,
                       inits = jags_inits,
                       parameters.to.save = jags_pars,
                       model.file = model_file,
                       n.chains = 3,
                       n.adapt = 100,

```

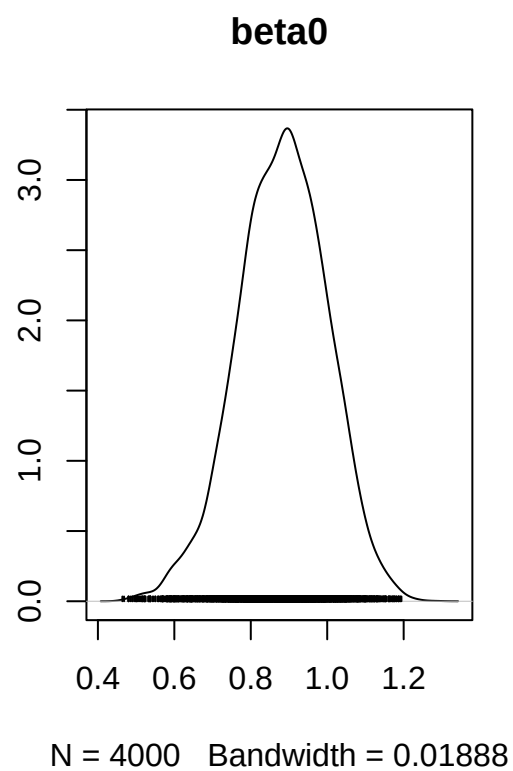
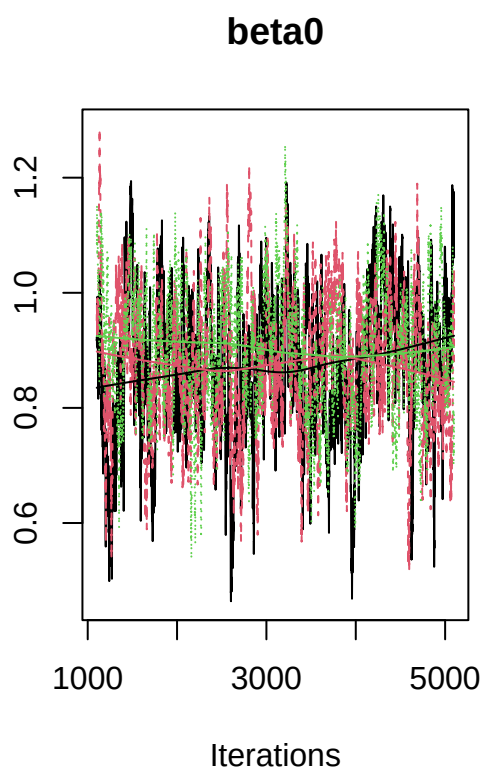
```
n.burnin = 1000,
n.iter = 5000,
parallel = TRUE)
```

```
##
## Processing function input.....
##
## Done.
##
## Beginning parallel processing using 3 cores. Console output will be suppressed.
##
## Parallel processing completed.
##
## MCMC took 0.179 minutes.
```

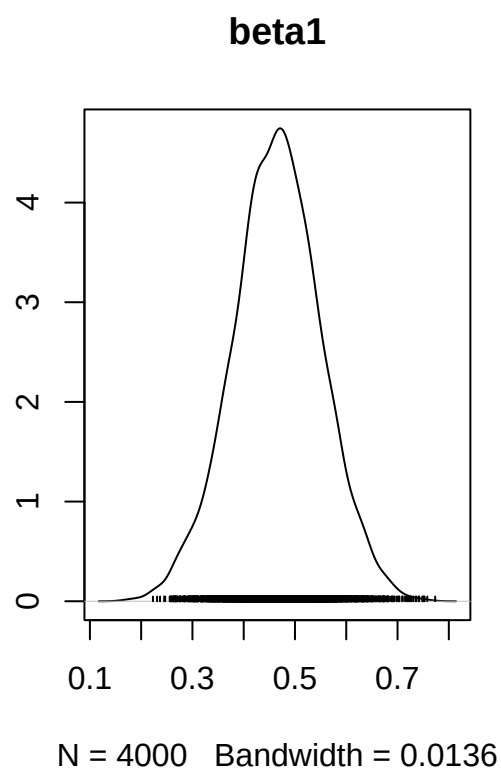
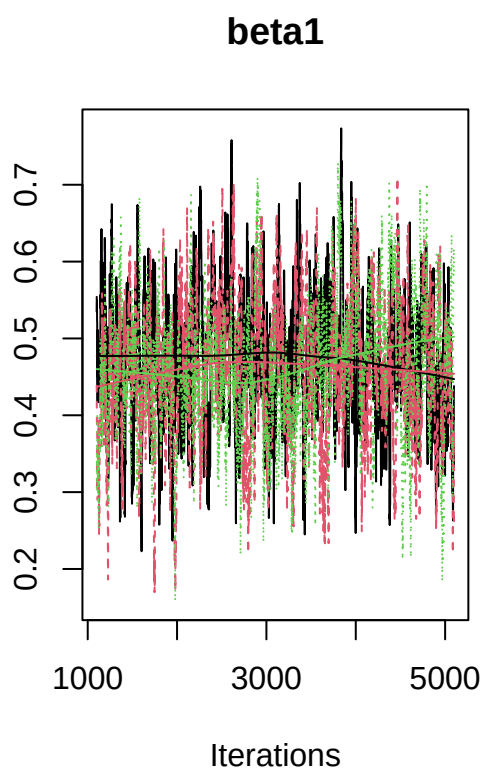
```
round(summary(jags_post)$quantile, digits=3)
```

```
##           2.5%      25%      50%      75%     97.5%
## alpha0      3.362    3.409    3.434    3.460    3.511
## alpha1     -0.028    0.010    0.030    0.051    0.092
## alpha2     -0.120   -0.088   -0.071   -0.054   -0.022
## beta0       0.641    0.805    0.886    0.964    1.100
## beta1       0.292    0.410    0.466    0.522    0.636
## beta2      -0.438   -0.322   -0.261   -0.199   -0.087
## beta3      -0.949   -0.828   -0.761   -0.698   -0.576
## deviance    883.842  902.256  912.119  922.268  942.089
## totalAbundance 1131.000 1276.000 1357.000 1440.000 1617.000
```

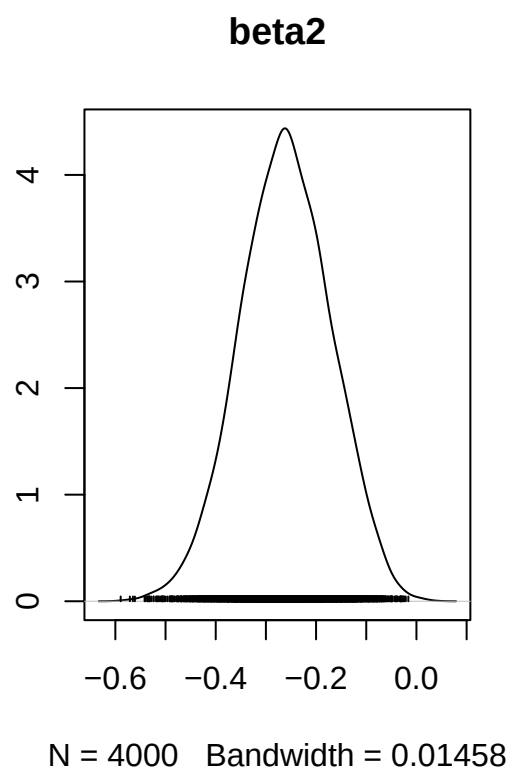
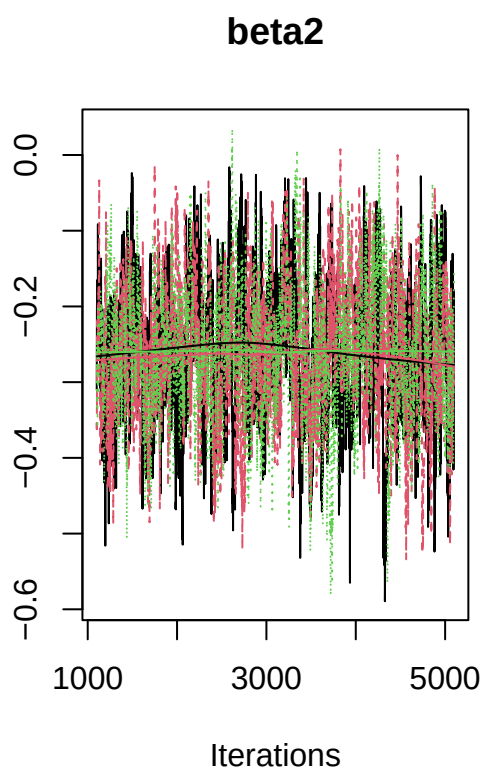
```
plot(jags_post[,jags_pars[1]], main = jags_pars[1])
```



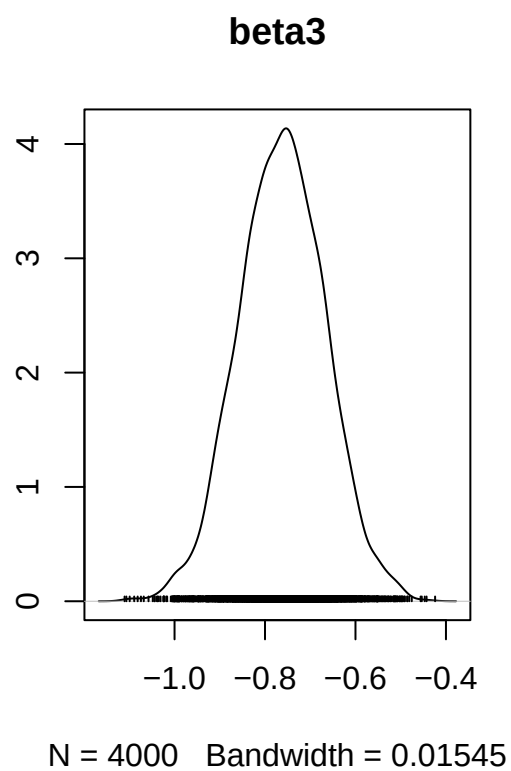
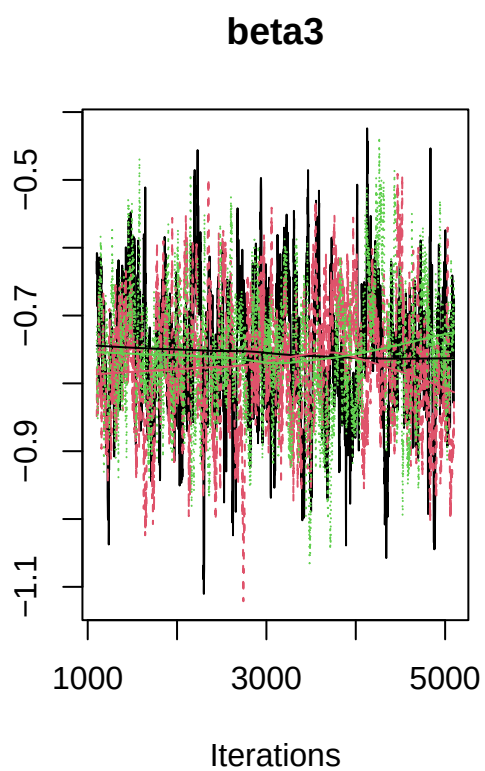
```
plot(jags_post[,jags_pars[2]], main = jags_pars[2])
```



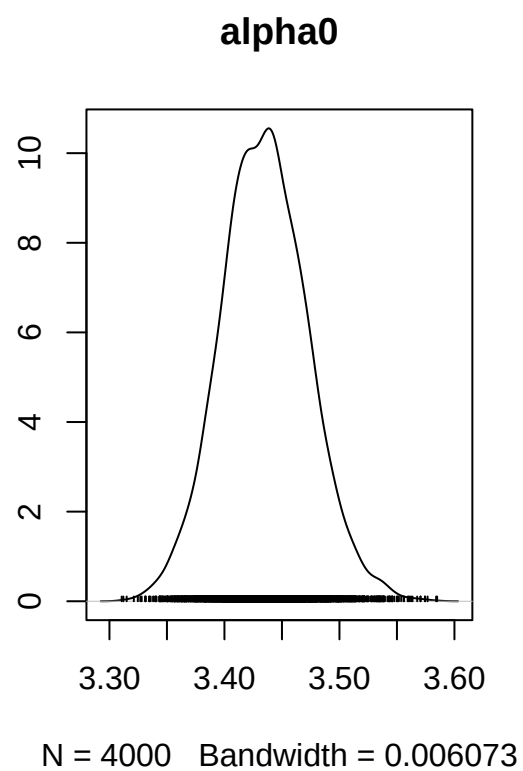
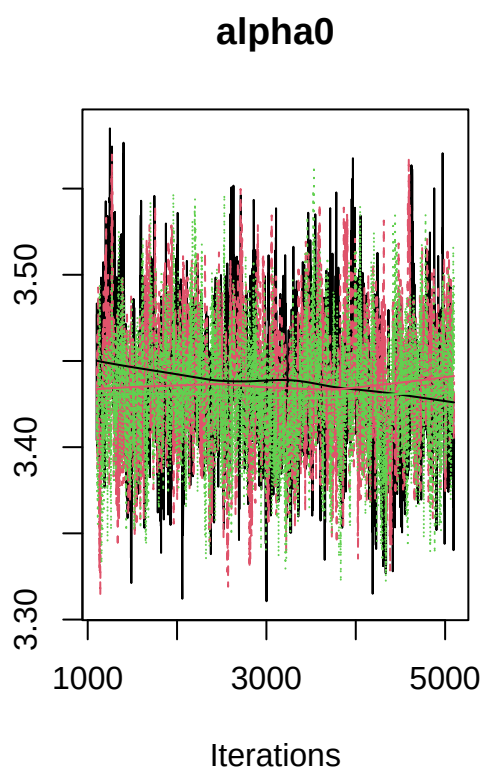
```
plot(jags_post[,jags_pars[3]], main = jags_pars[3])
```



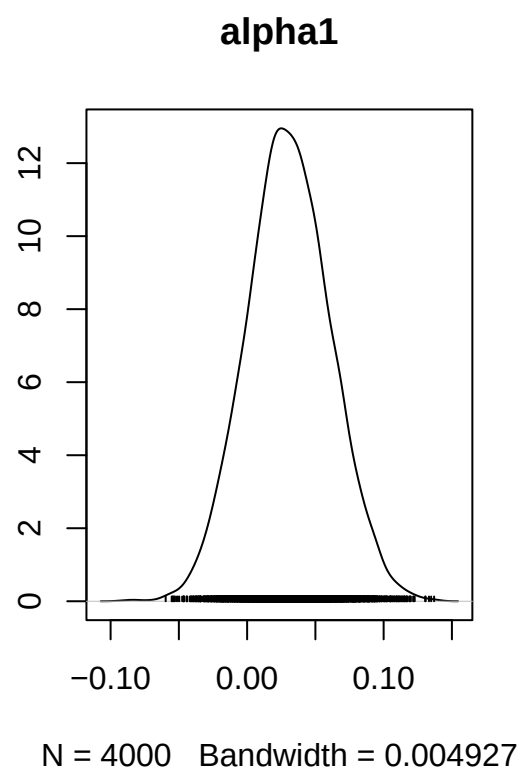
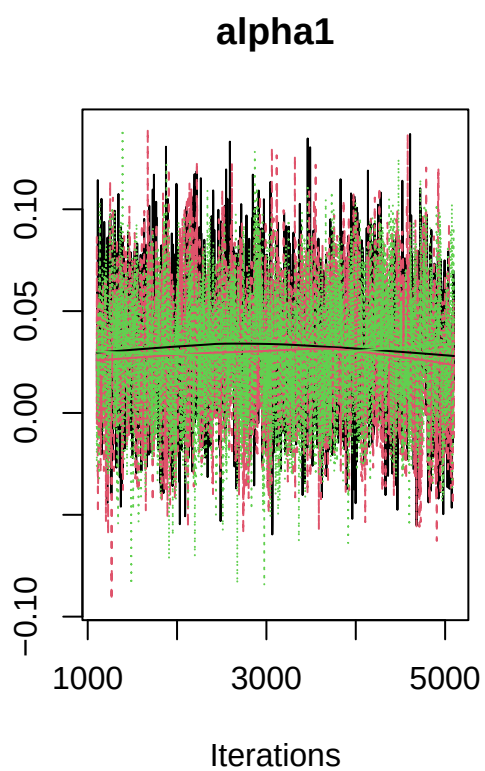
```
plot(jags_post[,jags_pars[4]], main = jags_pars[4])
```



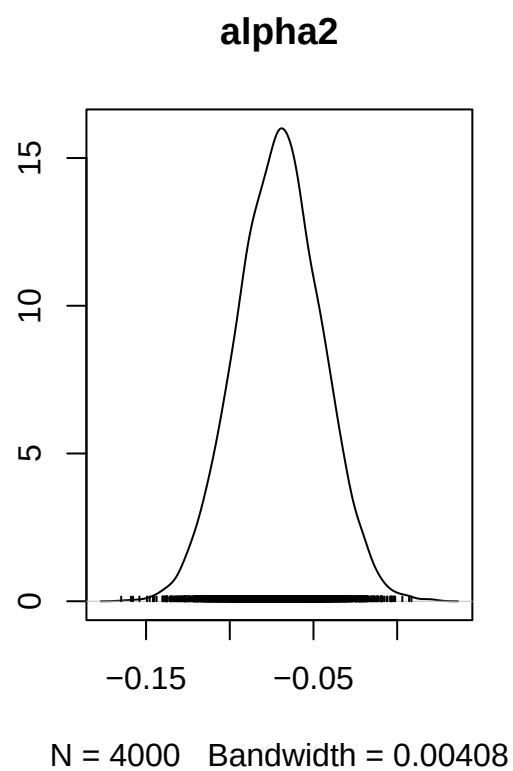
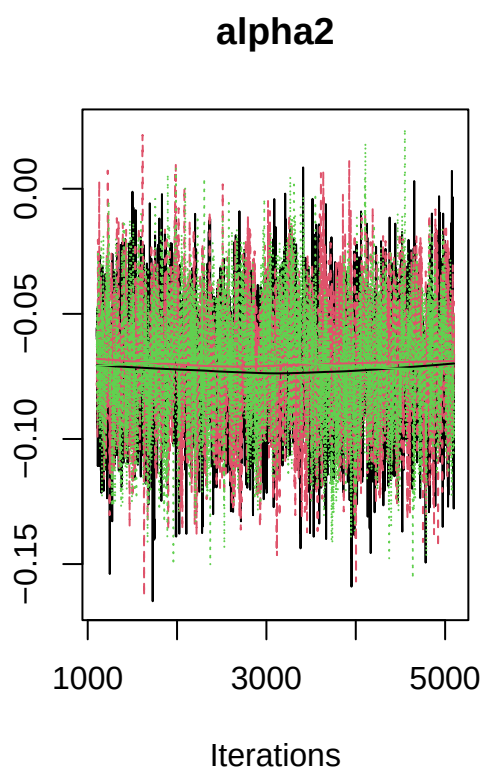
```
plot(jags_post[,jags_pars[5]], main = jags_pars[5])
```



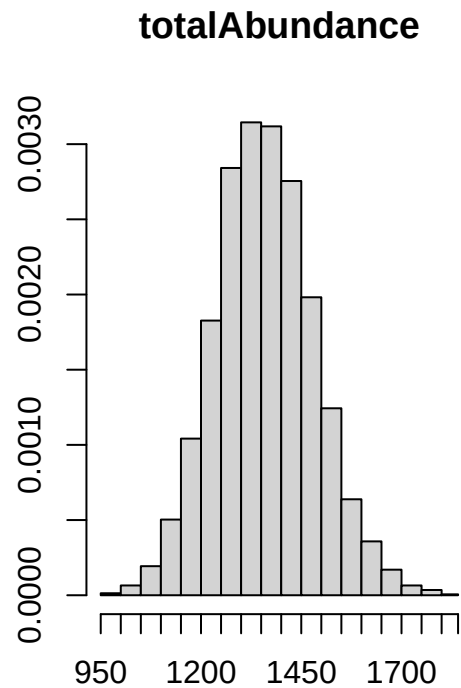
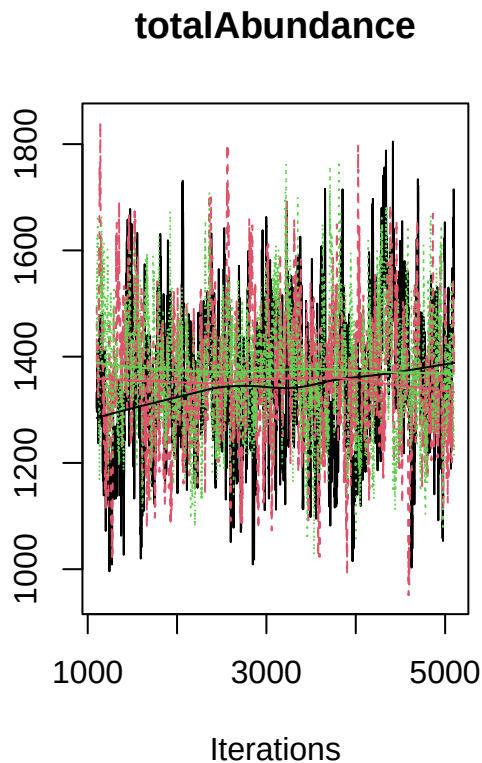
```
plot(jags_post[,jags_pars[6]], main = jags_pars[6])
```



```
plot(jags_post[,jags_pars[7]], main = jags_pars[7])
```



```
plot(jags_post[,jags_pars[8]], main = jags_pars[8])
```



unmarked prediction

```
library(ggplot2)

#density
pred_data <- data.frame(Elevation = seq(min(fm@data@siteCovs$Elevation),
                                         max(fm@data@siteCovs$Elevation),
                                         length.out = 100
                                         ),
                        UTM_N = mean(fm@data@siteCovs$UTM_N),
                        UTM_W = mean(fm@data@siteCovs$UTM_W)
                        )

pred_density <- predict(
  fm,
  type = "state",
  newdata = pred_data
)

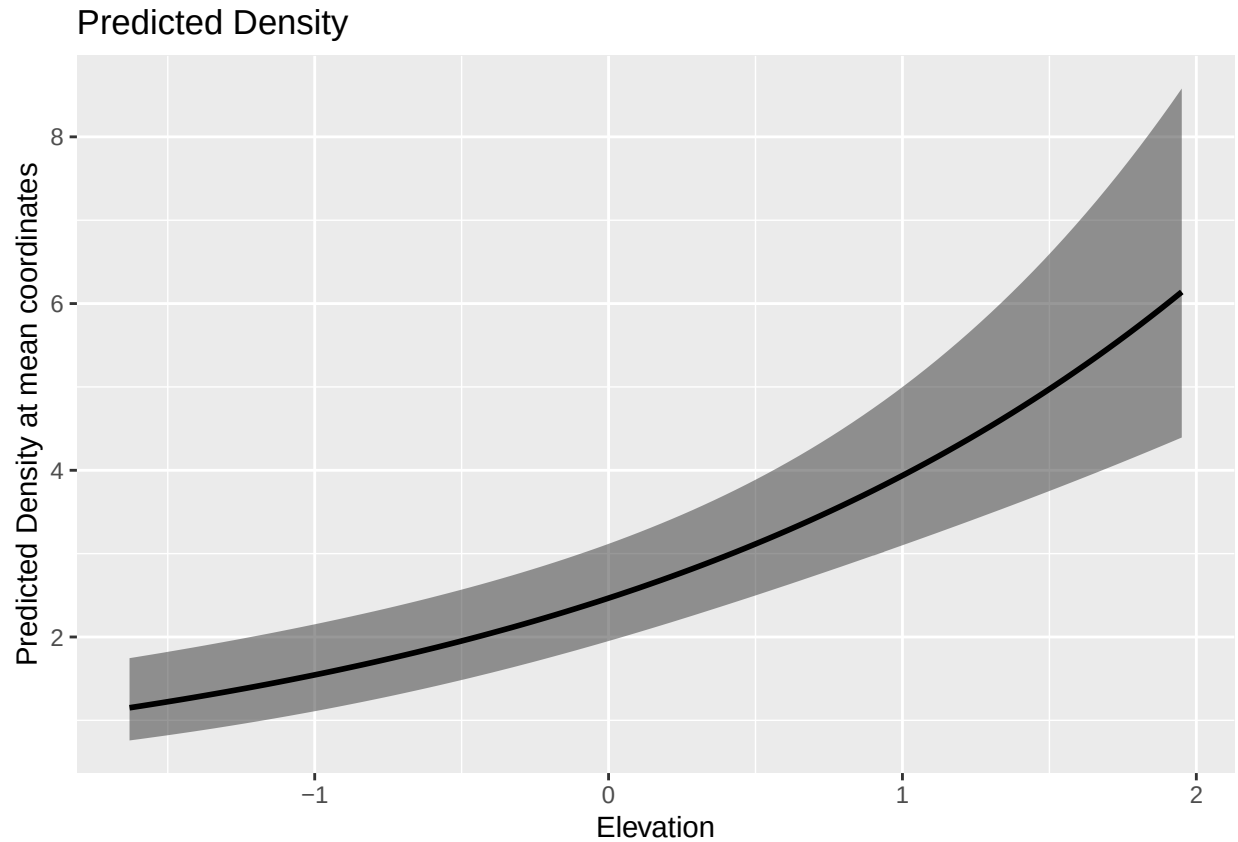
pred_data$Density <- pred_density$Predicted
pred_data$lower <- pred_density$lower
pred_data$upper <- pred_density$upper

ggplot(pred_data, aes(x=Elevation, y=Density)) +
  geom_ribbon(aes(ymin=lower, ymax=upper), alpha=0.5) +
  geom_line(linewidth=1) +
  labs(
```

```

x = "Elevation",
y = "Predicted Density at mean coordinates"
) +
ggtitle("Predicted Density")

```



```

pred_sigma_data <- data.frame(
  Noise = seq(min(fm@data@siteCovs$Noise),
              max(fm@data@siteCovs$Noise),
              length.out = 100
            ),
  Wind = mean(fm@data@siteCovs$Wind)
)

pred_sigma <- predict(
  fm,
  type="det",
  newdata = pred_sigma_data
)

pred_sigma_data$sigma <- pred_sigma$Predicted
pred_sigma_data$lower <- pred_sigma$lower
pred_sigma_data$upper <- pred_sigma$upper

ggplot(pred_sigma_data, aes(x=Noise, y=sigma)) +
  geom_ribbon(aes(ymin=lower, ymax=upper), alpha=0.5) +

```

```
geom_line(linewidth=1) +  
labs(  
  x = "Noise",  
  y = "Predicted sigma at mean wind"  
) +  
ggtitle("Predicted sigma")
```

